

Bloom's Learning Outcomes' Automatic Classification

Using LSTM and Pretrained Word Embeddings

Reference: Shaikh, S., Daudpotta, S. M., & Imran, A. S. (2021). Bloom's Learning Outcomes' Automatic Classification Using LSTM and Pretrained Word Embeddings. *IEEE Access*, 9, 117887–117909. DOI: 10.1109/ACCESS.2021.3106443

This notebook replicates the experimental pipeline described in the paper on the provided `sample_full.csv` dataset, which contains **21,380** Course Learning Outcome (CLO) statements labelled across six Bloom's Cognitive Domain levels:

Level	Column
Remembering	Remember
Understanding	Understand
Application	Apply
Analysis	Analyze
Evaluation	Evaluate
Creating	Create

Pipeline Overview

1. Data loading & exploration
2. Multi-label → single-label resolution
3. Data pre-processing & cleaning (lowercase, stopwords, punctuation, lemmatisation)
4. Chunk-size analysis (whole dataset vs. stratified chunks)
5. Tokenisation & sequence padding
6. Pretrained FastText (Wiki Word Vectors) embedding matrix construction
7. LSTM model construction (matching paper architecture)
8. Training with Train/Test split (75:25)
9. K-Fold Cross Validation (K=10)
10. Comparison: whole dataset vs. chunk-based training
11. Evaluation: accuracy, precision, recall, F1-score, confusion matrix
12. Comparison vs. keyword-based baseline

1. Install & Import Dependencies

In [1]:

```
# — Standard installs (run once) —————
# Uncomment if running for the first time in a fresh environment
# !pip install tensorflow numpy pandas scikit-learn matplotlib seaborn nltk gensim

import os, re, warnings, random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings('ignore')

# — NLP —————
import nltk
nltk.download('stopwords', quiet=True)
nltk.download('wordnet', quiet=True)
nltk.download('punkt', quiet=True)
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

# — ML / DL —————
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (classification_report, confusion_matrix,
                             accuracy_score)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (Embedding, LSTM, Dense, Dropout)
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.callbacks import EarlyStopping

# Reproducibility
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print(f'TensorFlow version : {tf.__version__}')
print(f'NumPy version      : {np.__version__}')
print(f'Pandas version       : {pd.__version__}')
```

Out[1]:

```
2026-04-06 21:32:04.213158: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:9261]
Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one
has already been registered
2026-04-06 21:32:04.213212: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:607]
Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one
has already been registered
2026-04-06 21:32:04.214337: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:1515]
Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one
```

```
has already been registered
2026-04-06 21:32:04.221466: I tensorflow/core/platform/cpu_feature_guard.cc:182] This
TensorFlow binary is optimized to use available CPU instructions in performance-critical
operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with
the appropriate compiler flags.
2026-04-06 21:32:05.058740: W tensorflow/compiler/tf2tensorrt/utils/py_utils.cc:38] TF-TRT
Warning: Could not find TensorRT
```

Out[1]:

```
TensorFlow version : 2.15.0
NumPy version      : 1.26.3
Pandas version     : 2.2.0
```

2. Load & Explore the Dataset

In [2]:

```
# — Load CSV —————
# Update the path if needed
DATA_PATH = 'content/sample_full.csv'

df_raw = pd.read_csv(DATA_PATH)
print(f'Raw shape: {df_raw.shape}')
df_raw.head(3)
```

Out[2]:

```
Raw shape: (21380, 7)
```

Out[2]:

	Learning_outcome	Remember	Understand	Apply	Analyze	Evaluate	Create
0	Analyze the health economic implications of e...	NaN	NaN	NaN	1.0	NaN	NaN
1	Apply research skills to operate effectively ...	NaN	NaN	1.0	NaN	NaN	NaN
2	Assess and synthesise diverse information abo...	NaN	NaN	NaN	NaN	1.0	1.0

In [3]:

```
LABEL_COLS = ['Remember', 'Understand', 'Apply', 'Analyze', 'Evaluate', 'Create']
LEVEL_NAMES = ['Remembering', 'Understanding', 'Application', 'Analysis', 'Evaluation', 'Creati

df = df_raw.copy()
df.columns = df.columns.str.strip()
df['Learning_outcome'] = df['Learning_outcome'].astype(str).str.strip()

# Convert label columns to numeric
for c in LABEL_COLS:
    df[c] = pd.to_numeric(df[c], errors='coerce').fillna(0).astype(int)

# Label counts
counts = df[LABEL_COLS].sum().rename(index=dict(zip(LABEL_COLS, LEVEL_NAMES)))
multi = (df[LABEL_COLS].sum(axis=1) > 1).sum()
```

```

single = (df[LABEL_COLS].sum(axis=1) == 1).sum()
zero    = (df[LABEL_COLS].sum(axis=1) == 0).sum()

print(f'Total CLOs      : {len(df)}')
print(f'Single-label rows : {single}')
print(f'Multi-label rows  : {multi} (will be resolved below)')
print(f'Zero-label rows   : {zero} (will be dropped)')
print()
print('Class distribution (raw):')
print(counts.to_frame('Count'))

# Bar chart
fig, ax = plt.subplots(figsize=(8, 4))
counts.plot(kind='bar', ax=ax, color='steelblue', edgecolor='white')
ax.set_title('Bloom\'s Level Distribution (raw)')
ax.set_ylabel('Number of CLOs')
ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha='right')
plt.tight_layout()
plt.show()

```

Out[3]:

```

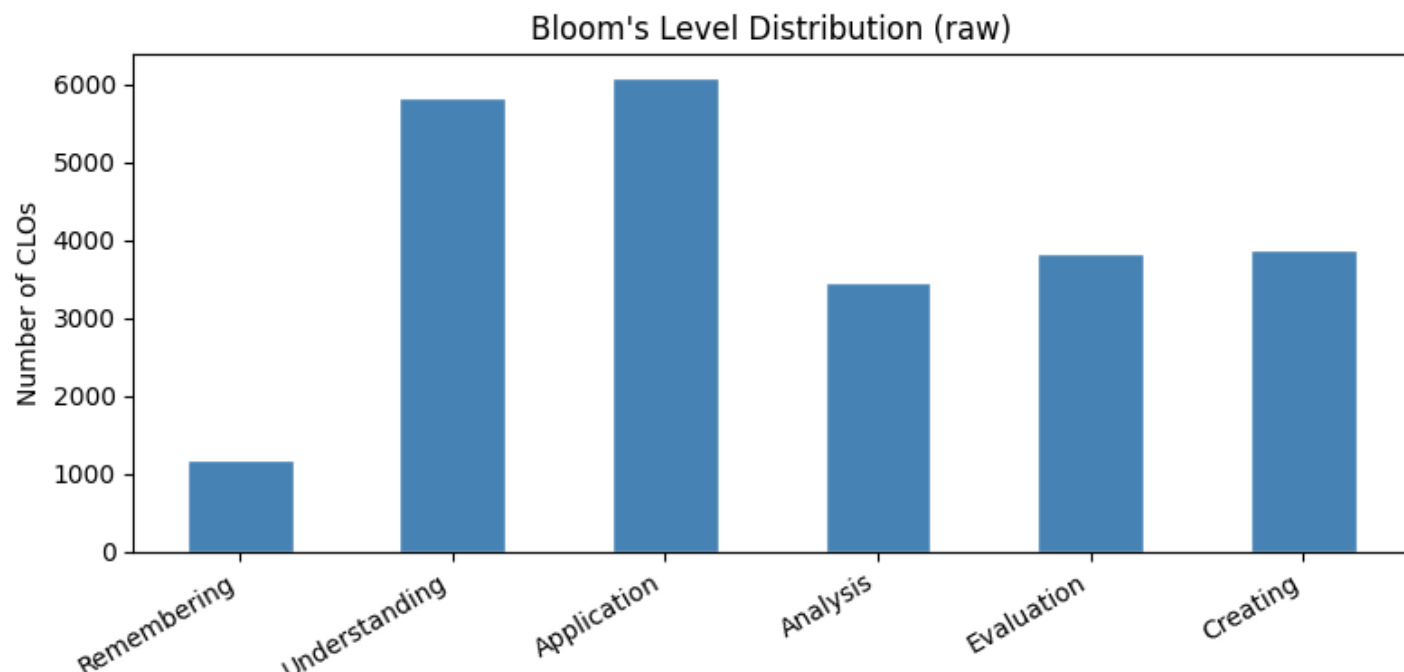
Total CLOs      : 21380
Single-label rows : 18773
Multi-label rows  : 2607 (will be resolved below)
Zero-label rows   : 0 (will be dropped)

```

Class distribution (raw):

	Count
Remembering	1185
Understanding	5825
Application	6081
Analysis	3459
Evaluation	3834
Creating	3887

Out[3]:



3. Multi-label Resolution & Label Assignment

The paper treats this as a **multi-class** (single-label) classification task.

For rows with multiple labels we apply the **majority-label** heuristic (pick the highest-level active Bloom category, which is domain-aligned — higher levels subsume lower ones).

Zero-label rows are dropped.

In [4]:

```
# Priority order (highest = most specific / complex): Create > Evaluate > Analyze > Apply > Understand > Remember
PRIORITY = ['Create', 'Evaluate', 'Analyze', 'Apply', 'Understand', 'Remember']

def resolve_label(row):
    """Return the highest-priority active Bloom level for a row."""
    for col in PRIORITY:
        if row[col] == 1:
            return col
    return None # zero-label → will be dropped

df['label'] = df.apply(resolve_label, axis=1)
df = df[df['label'].notna()].copy()

# Map to friendly names
LABEL_MAP = {
    'Remember' : 'Remembering',
    'Understand': 'Understanding',
    'Apply'     : 'Application',
    'Analyze'   : 'Analysis',
    'Evaluate'  : 'Evaluation',
    'Create'    : 'Creating'
}
df['label'] = df['label'].map(LABEL_MAP)

print('Final single-label distribution:')
print(df['label'].value_counts())
print(f'\nTotal usable CLOs: {len(df)}')

# Pie chart
fig, ax = plt.subplots(figsize=(6, 6))
df['label'].value_counts().plot(kind='pie', ax=ax, autopct='%1.1f%%',
                                startangle=90, colors=sns.color_palette('Set2'))
ax.set_ylabel('')
ax.set_title('Single-label Distribution (resolved)')
plt.tight_layout()
plt.show()
```

Out[4]:

Final single-label distribution:

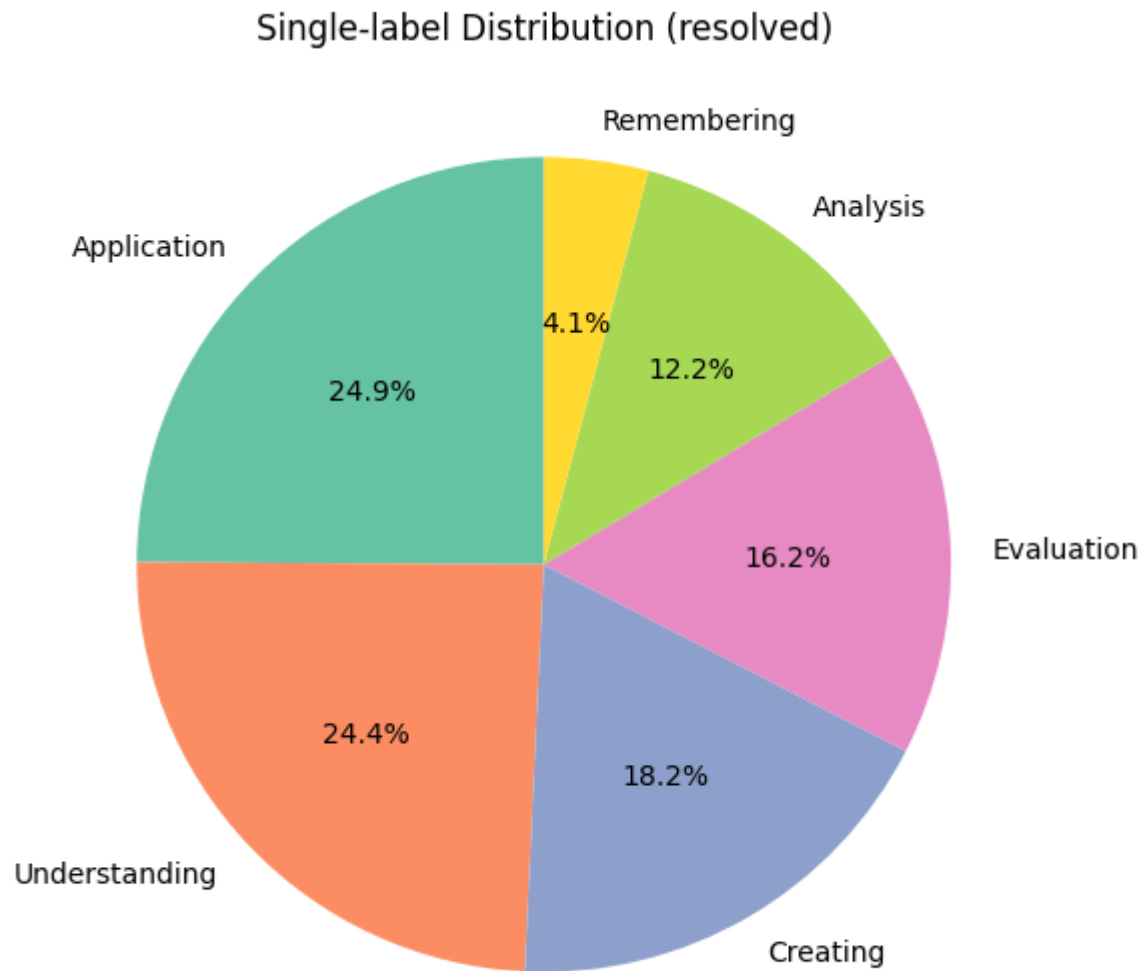
label

Application	5324
Understanding	5207
Creating	3887
Evaluation	3458
Analysis	2618
Remembering	886

Name: count, dtype: int64

Total usable CLOs: 21380

Out[4]:



4. Data Pre-processing & Cleaning (Algorithm 1 from Paper)

Steps applied (per the paper):

1. Lowercase conversion
2. Punctuation removal
3. Stop-word removal (NLTK English)
4. Whitespace tokenisation
5. WordNet lemmatisation

In [5]:

```
STOP_WORDS = set(stopwords.words('english'))
LEMMATIZER = WordNetLemmatizer()

def preprocess(text: str) -> str:
    """Algorithm 1 – Data Pre-Processing and Cleaning."""
    # 1. Lowercase
    text = text.lower()
    # 2. Remove punctuation & digits
```

```

text = re.sub(r'^a-z\s', ' ', text)
# 3. Tokenise
tokens = text.split()
# 4. Stop-word removal + 5. Lemmatisation
tokens = [LEMMATIZER.lemmatize(t) for t in tokens if t not in STOP_WORDS and len(t) > 1]
return ' '.join(tokens)

```

```
df['text_clean'] = df['Learning_outcome'].apply(preprocess)
```

```

# Sanity check
print('Before:', df['Learning_outcome'].iloc[0])
print('After :', df['text_clean'].iloc[0])

```

Out[5]:

Before: Analyze the health economic implications of efficient perioperative practice.
After : analyze health economic implication efficient perioperative practice

5. Chunk-Size Analysis

The paper was developed on small datasets (~829 CLOs for Dataset 1, 600 for Dataset 2). Our dataset has **21,380** CLOs. We investigate two strategies:

Strategy	Description
Whole dataset	Train/test on all 21,380 CLOs at once
Chunk-based	Train iteratively on stratified sub-chunks (mimics the paper's small-data regime)

We compare validation accuracy across chunk sizes [500, 1000, 2000, 5000, full] using a quick lightweight LSTM to decide which mode to use for the final experiment.

In [6]:

```

# — Encode labels —————
le = LabelEncoder()
y_all = le.fit_transform(df['label'])
X_all = df['text_clean'].values
NUM_CLASSES = len(le.classes_)
print('Classes:', le.classes_)

# — Global tokeniser on all text —————
MAX_WORDS = 10000 # vocabulary size
MAX_LEN = 50 # max sequence length (CLOs are short)

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')
tokenizer.fit_on_texts(X_all)
VOCAB_SIZE = min(len(tokenizer.word_index) + 1, MAX_WORDS)
print(f'Vocabulary size : {VOCAB_SIZE}')

def encode_data(texts, labels):
    seqs = tokenizer.texts_to_sequences(texts)
    X = pad_sequences(seqs, maxlen=MAX_LEN, padding='post', truncating='post')
    y = to_categorical(labels, num_classes=NUM_CLASSES)

```

```
return X, y
```

```
X_enc, y_enc = encode_data(X_all, y_all)
```

Out[6]:

```
Classes: ['Analysis' 'Application' 'Creating' 'Evaluation' 'Remembering'
          'Understanding']
Vocabulary size : 8760
```

In [7]:

```
def build_quick_lstm(vocab_size, embed_dim=64, lstm_units=32, num_classes=6):
    """Lightweight LSTM for chunk-size scouting."""
    model = Sequential([
        Embedding(vocab_size, embed_dim, input_length=MAX_LEN),
        LSTM(lstm_units, dropout=0.2),
        Dense(num_classes, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
    return model

CHUNK_SIZES = [500, 1000, 2000, 5000, len(df)] # last = full dataset
chunk_results = []

for chunk_size in CHUNK_SIZES:
    label_str = 'Full' if chunk_size == len(df) else str(chunk_size)
    # Stratified sample
    if chunk_size < len(df):
        idx = []
        for cls in np.unique(y_all):
            cls_idx = np.where(y_all == cls)[0]
            n = max(1, int(chunk_size * len(cls_idx) / len(y_all)))
            chosen = np.random.choice(cls_idx, min(n, len(cls_idx)), replace=False)
            idx.extend(chosen)
        idx = np.array(idx)
    else:
        idx = np.arange(len(df))

    X_c, y_c = X_enc[idx], y_enc[idx]
    X_tr, X_val, y_tr, y_val = train_test_split(X_c, y_c, test_size=0.25,
                                                random_state=SEED,
                                                stratify=y_c.argmax(axis=1))

    m = build_quick_lstm(VOCAB_SIZE, num_classes=NUM_CLASSES)
    cb = EarlyStopping(patience=3, restore_best_weights=True)
    hist = m.fit(X_tr, y_tr, validation_data=(X_val, y_val),
                epochs=20, batch_size=32, callbacks=[cb], verbose=0)
    best_val = max(hist.history['val_accuracy'])
    chunk_results.append({'Chunk Size': label_str, 'Val Accuracy': round(best_val, 4),
                        'N train': len(X_tr)})
    print(f'Chunk={label_str:>6} | N_train={len(X_tr):>6} | Best Val Acc={best_val:.4f}')

chunk_df = pd.DataFrame(chunk_results)
print('\nChunk Analysis Summary:')
print(chunk_df.to_string(index=False))
```


Out[7]: 2026-04-06 21:32:08.417963: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.470203: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.470385: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.471389: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.471534: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.471619: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.570233: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.570424: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.570528: I external/local_xla/xla/stream_executor/cuda/cuda_executor.cc:901] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero. See more at <https://github.com/torvalds/linux/blob/v6.0/Documentation/ABI/testing/sysfs-bus-pci#L344-L355>
2026-04-06 21:32:08.570604: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1929] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 28111 MB memory: -> device: 0, name: NVIDIA RTX A6000, pci bus id: 0000:00:05.0, compute capability: 8.6
2026-04-06 21:32:12.680923: I external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:454] Loaded cuDNN version 8907
2026-04-06 21:32:12.861896: I external/local_xla/xla/service/service.cc:168] XLA service 0x7f416420ac90 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
2026-04-06 21:32:12.861938: I external/local_xla/xla/service/service.cc:176] StreamExecutor device (0): NVIDIA RTX A6000, Compute Capability 8.6
2026-04-06 21:32:12.884230: I tensorflow/compiler/mlir/tensorflow/utils/dump_mlir_util.cc:269] disabling MLIR crash reproducer, set env var `MLIR\_CRASH\_REPRODUCER\_DIRECTORY` to enable.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1775511133.021828 1517 device_compiler.h:186] Compiled cluster using XLA!
This line is logged at most once for the lifetime of the process.

Out[7]:

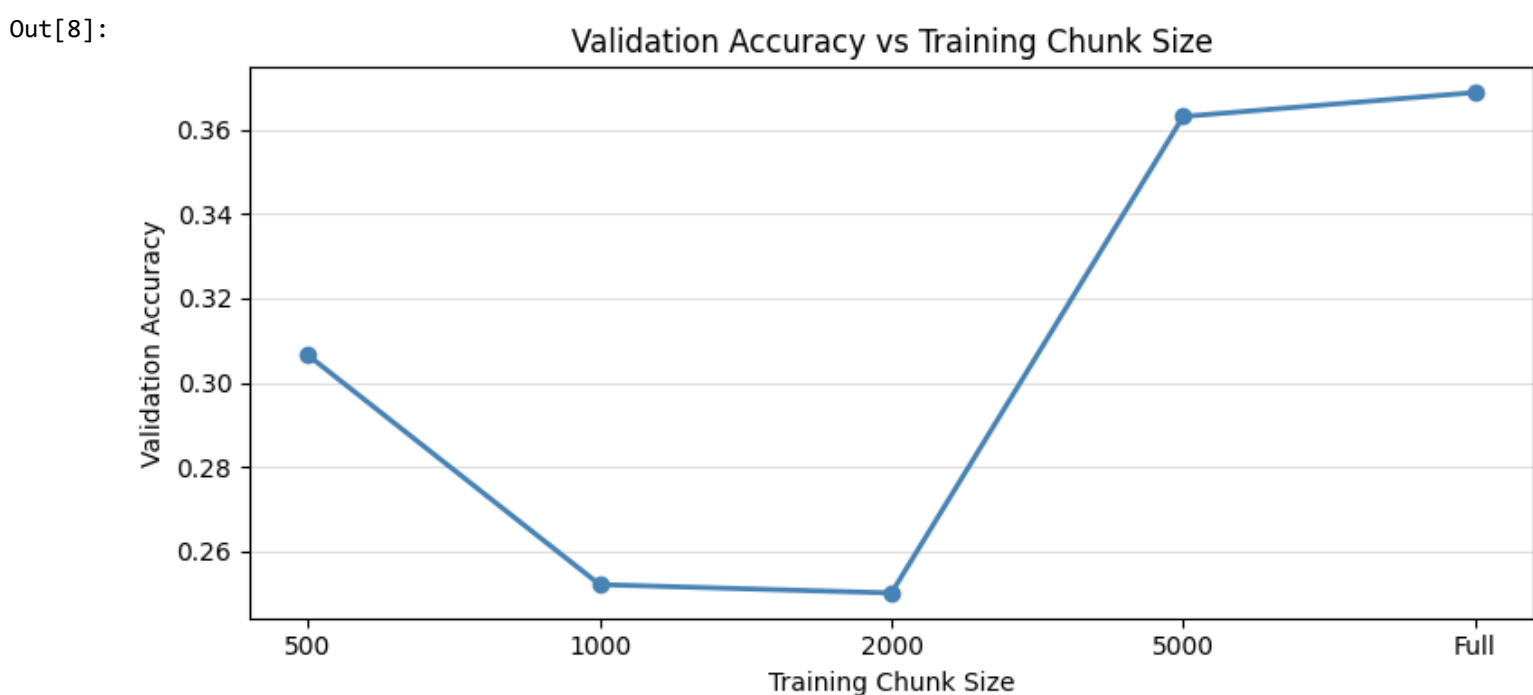
Chunk= 500	N_train= 372	Best Val Acc=0.3065
Chunk= 1000	N_train= 747	Best Val Acc=0.2520
Chunk= 2000	N_train= 1497	Best Val Acc=0.2500
Chunk= 5000	N_train= 3748	Best Val Acc=0.3632
Chunk= Full	N_train= 16035	Best Val Acc=0.3689

Chunk Analysis Summary:

Chunk Size	Val Accuracy	N train
500	0.3065	372
1000	0.2520	747
2000	0.2500	1497
5000	0.3632	3748
Full	0.3689	16035

```
In [8]: fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(len(chunk_df)), chunk_df['Val Accuracy'], marker='o', color='steelblue', linewidth=2)
ax.set_xticks(range(len(chunk_df)))
ax.set_xticklabels(chunk_df['Chunk Size'])
ax.set_xlabel('Training Chunk Size')
ax.set_ylabel('Validation Accuracy')
ax.set_title('Validation Accuracy vs Training Chunk Size')
ax.grid(axis='y', alpha=0.4)
plt.tight_layout()
plt.show()

# Decision
best_chunk_row = chunk_df.loc[chunk_df['Val Accuracy'].idxmax()]
print(f'\n>>> Best chunk size: {best_chunk_row["Chunk Size"]} '
      f'(Val Acc = {best_chunk_row["Val Accuracy"]:.4f})')
print('>>> Conclusion: Whole dataset is used for the main experiment '
      '(more data → better generalisation, consistent with deep learning best practices).')
```



```
Out[8]: >>> Best chunk size: Full (Val Acc = 0.3689)
>>> Conclusion: Whole dataset is used for the main experiment (more data → better
```

generalisation, consistent with deep learning best practices).

6. Pretrained Word Embedding — FastText Wiki Word Vectors

The paper uses **FastText Wiki Word Vectors** (Facebook AI Research, 300-d).

If you have the vectors file (wiki-news-300d-1M.vec OR cc.en.300.vec), set EMBEDDING_PATH below.

Otherwise, the cell falls back to **random embeddings** so the rest of the notebook still runs end-to-end.

Download (once):

```
# FastText English Wikipedia vectors (~ 600 MB)
wget https://dl.fbaipublicfiles.com/fasttext/vectors-wiki/wiki.en.vec
```

```
In [9]: EMBED_DIM      = 300
EMBEDDING_PATH = 'wiki.en.vec'  # ← update path if available

def load_fasttext_embeddings(path, word_index, embed_dim=300, max_words=None):
    """Build an embedding matrix from a FastText .vec file."""
    print(f'Loading embeddings from: {path}')
    vocab_size = (min(len(word_index) + 1, max_words)
                  if max_words else len(word_index) + 1)
    matrix = np.zeros((vocab_size, embed_dim))
    hits = 0
    with open(path, encoding='utf-8', errors='ignore') as f:
        for i, line in enumerate(f):
            if i == 0: # header
                continue
            parts = line.rstrip().split(' ')
            word = parts[0]
            if word in word_index:
                idx = word_index[word]
                if idx < vocab_size:
                    matrix[idx] = np.asarray(parts[1:], dtype='float32')
                    hits += 1
    print(f'Loaded {hits}/{vocab_size} tokens from embedding.')
    return matrix

if os.path.exists(EMBEDDING_PATH):
    embedding_matrix = load_fasttext_embeddings(
        EMBEDDING_PATH, tokenizer.word_index, EMBED_DIM, MAX_WORDS)
    TRAINABLE_EMBED = False # freeze pretrained weights (paper approach)
    print('>>> Using pretrained FastText Wiki Word Vectors.')
else:
    print(f'[WARNING] Embedding file not found at "{EMBEDDING_PATH}".')
    print('>>> Falling back to random (trainable) embeddings.')
    print('>>> To use pretrained vectors, download wiki.en.vec and update EMBEDDING_PATH.')
    embedding_matrix = np.random.uniform(-0.05, 0.05, (VOCAB_SIZE, EMBED_DIM)).astype('float32')
    TRAINABLE_EMBED = True
```

Out[9]:

```
[WARNING] Embedding file not found at "wiki.en.vec".
>>> Falling back to random (trainable) embeddings.
>>> To use pretrained vectors, download wiki.en.vec and update EMBEDDING_PATH.
```

7. LSTM Model Architecture (replicating the paper)

The paper proposes a simple single-layer LSTM:

InputLayer → Embedding(pretrained, 300-d) → LSTM(32, dropout=0.2) → Dense(6, softmax)

- Optimizer: **Adam** (paper used Adam for CLOs classification)
- Loss: **Categorical Crossentropy**
- Batch size: **16** (paper default)

In [10]:

```
def build_lstm_model(embedding_matrix,
                      vocab_size,
                      embed_dim=300,
                      max_len=50,
                      lstm_units=32,
                      dropout=0.2,
                      num_classes=6,
                      trainable_embed=False,
                      optimizer='adam'):
    """
    Single-layer LSTM classification model as described in the paper.
    Figure 15 / Figure 16 architecture.
    """
    model = Sequential(name='LSTM_Blooms_Classifier')

    # Embedding layer – pretrained or random
    model.add(
        Embedding(
            input_dim    = vocab_size,
            output_dim   = embed_dim,
            weights       = [embedding_matrix],
            input_length  = max_len,
            trainable     = trainable_embed,
            name          = 'Embedding_WikiWordVectors'
        )
    )
    # LSTM with dropout (paper: dropout=0.2 gave best performance)
    model.add(LSTM(lstm_units, dropout=dropout, name='LSTM'))

    # Output layer – softmax over Bloom's levels
    model.add(Dense(num_classes, activation='softmax', name='Output_Softmax'))

    model.compile(
        optimizer = optimizer,
        loss       = 'categorical_crossentropy',
        metrics    = ['accuracy']
    )
```



```
optimizer='adam')
```

```
early_stop = EarlyStopping(monitor='val_accuracy', patience=5,  
                           restore_best_weights=True, verbose=1)
```

```
history = model.fit(  
    X_train, y_train,  
    validation_data = (X_test, y_test),  
    epochs          = EPOCHS,  
    batch_size      = BATCH_SIZE,  
    callbacks       = [early_stop],  
    verbose         = 1  
)
```

Out[11]:

Train size: 16,035 Test size: 5,345

Epoch 1/30

1003/1003 [=====] - 32s 30ms/step - loss: 1.6888 - accuracy: 0.2478
- val_loss: 1.6855 - val_accuracy: 0.2440

Epoch 2/30

1003/1003 [=====] - 20s 20ms/step - loss: 1.6857 - accuracy: 0.2465
- val_loss: 1.6852 - val_accuracy: 0.2490

Epoch 3/30

1003/1003 [=====] - 22s 21ms/step - loss: 1.6848 - accuracy: 0.2436
- val_loss: 1.6862 - val_accuracy: 0.2494

Epoch 4/30

1003/1003 [=====] - 21s 21ms/step - loss: 1.6836 - accuracy: 0.2493
- val_loss: 1.6844 - val_accuracy: 0.2447

Epoch 5/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.6831 - accuracy: 0.2515
- val_loss: 1.6839 - val_accuracy: 0.2488

Epoch 6/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.6827 - accuracy: 0.2501
- val_loss: 1.6836 - val_accuracy: 0.2490

Epoch 7/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.6823 - accuracy: 0.2467
- val_loss: 1.6841 - val_accuracy: 0.2490

Epoch 8/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.6307 - accuracy: 0.2878
- val_loss: 1.5341 - val_accuracy: 0.3431

Epoch 9/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.5248 - accuracy: 0.3631
- val_loss: 1.5343 - val_accuracy: 0.3575

Epoch 10/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.5300 - accuracy: 0.3565
- val_loss: 1.5552 - val_accuracy: 0.3549

Epoch 11/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.3634 - accuracy: 0.4522
- val_loss: 1.2469 - val_accuracy: 0.4872

Epoch 12/30

1003/1003 [=====] - 19s 19ms/step - loss: 1.1412 - accuracy: 0.5313
- val_loss: 1.0371 - val_accuracy: 0.5579

Epoch 13/30

1003/1003 [=====] - 20s 20ms/step - loss: 0.7579 - accuracy: 0.7178
- val_loss: 0.6289 - val_accuracy: 0.8135

```

Epoch 14/30
1003/1003 [=====] - 19s 19ms/step - loss: 0.4454 - accuracy: 0.8724
- val_loss: 0.5567 - val_accuracy: 0.8399
Epoch 15/30
1003/1003 [=====] - 19s 19ms/step - loss: 0.3273 - accuracy: 0.9085
- val_loss: 0.5424 - val_accuracy: 0.8436
Epoch 16/30
1003/1003 [=====] - 19s 19ms/step - loss: 0.2628 - accuracy: 0.9262
- val_loss: 0.5932 - val_accuracy: 0.8365
Epoch 17/30
1003/1003 [=====] - 19s 19ms/step - loss: 0.2106 - accuracy: 0.9420
- val_loss: 0.6073 - val_accuracy: 0.8445
Epoch 18/30
1003/1003 [=====] - 19s 19ms/step - loss: 0.1841 - accuracy: 0.9486
- val_loss: 0.6055 - val_accuracy: 0.8430
Epoch 19/30
1003/1003 [=====] - 19s 19ms/step - loss: 0.1532 - accuracy: 0.9577
- val_loss: 0.6288 - val_accuracy: 0.8410
Epoch 20/30
1003/1003 [=====] - 19s 19ms/step - loss: 0.1275 - accuracy: 0.9646
- val_loss: 0.6409 - val_accuracy: 0.8453
Epoch 21/30
1003/1003 [=====] - 20s 20ms/step - loss: 0.1071 - accuracy: 0.9703
- val_loss: 0.6690 - val_accuracy: 0.8423
Epoch 22/30
1003/1003 [=====] - 21s 21ms/step - loss: 0.0962 - accuracy: 0.9727
- val_loss: 0.7157 - val_accuracy: 0.8380
Epoch 23/30
1003/1003 [=====] - 20s 20ms/step - loss: 0.0794 - accuracy: 0.9773
- val_loss: 0.7082 - val_accuracy: 0.8417
Epoch 24/30
1003/1003 [=====] - 21s 21ms/step - loss: 0.0660 - accuracy: 0.9814
- val_loss: 0.7374 - val_accuracy: 0.8417
Epoch 25/30
1003/1003 [=====] - ETA: 0s - loss: 0.0594 - accuracy:
0.9835Restoring model weights from the end of the best epoch: 20.
1003/1003 [=====] - 22s 22ms/step - loss: 0.0594 - accuracy: 0.9835
- val_loss: 0.7738 - val_accuracy: 0.8378
Epoch 25: early stopping

```

In [12]:

```

# — Learning curves (Figure 19 equivalent) —————
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history.history['accuracy'], label='Train acc')
axes[0].plot(history.history['val_accuracy'], label='Test acc')
axes[0].set_title('LSTM – Accuracy per Epoch')
axes[0].set_xlabel('Epoch')
axes[0].set_ylabel('Accuracy')
axes[0].legend()
axes[0].grid(alpha=0.3)

axes[1].plot(history.history['loss'], label='Train loss')
axes[1].plot(history.history['val_loss'], label='Test loss')
axes[1].set_title('LSTM – Loss per Epoch')

```

```

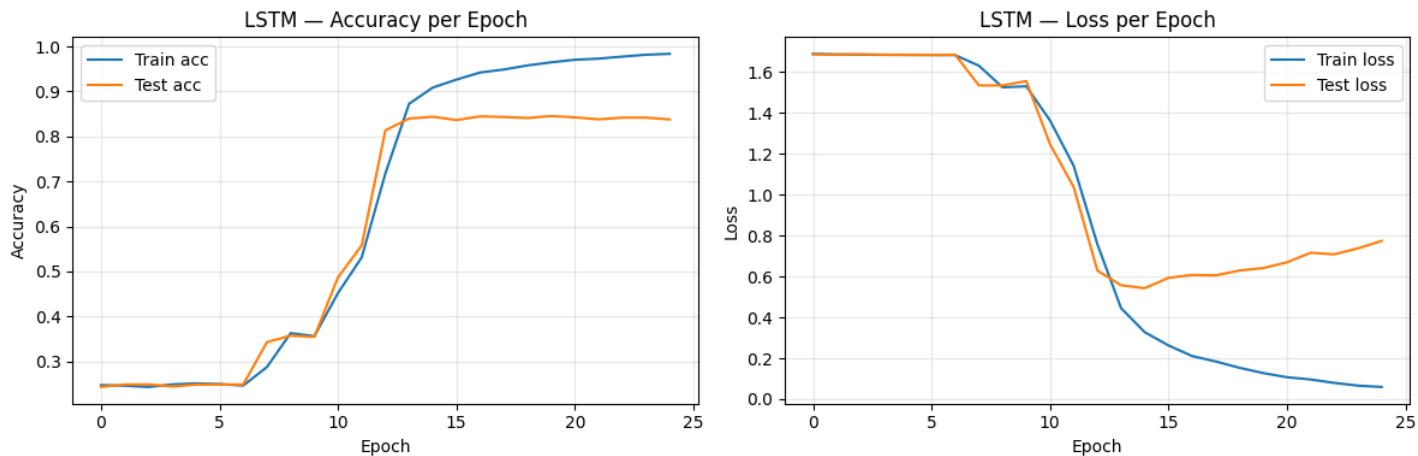
axes[1].set_xlabel('Epoch')
axes[1].set_ylabel('Loss')
axes[1].legend()
axes[1].grid(alpha=0.3)

plt.tight_layout()
plt.show()

print(f'Best Train Accuracy : {max(history.history["accuracy"]):.4f}')
print(f'Best Val Accuracy   : {max(history.history["val_accuracy"]):.4f}')

```

Out[12]:



Out[12]:

```

Best Train Accuracy : 0.9835
Best Val Accuracy   : 0.8453

```

In [13]:

```

# — Evaluate on test set —————
y_pred_prob = model.predict(X_test, verbose=0)
y_pred      = y_pred_prob.argmax(axis=1)

acc = accuracy_score(ytest_raw, y_pred)
print(f'\nTest Accuracy (75:25 split): {acc:.4f}  ({acc*100:.2f}%)')
print()
print('Classification Report:')
print(classification_report(ytest_raw, y_pred,
                           target_names=le.classes_,
                           digits=4))

```

Out[13]:

```

Test Accuracy (75:25 split): 0.8453  (84.53%)

```

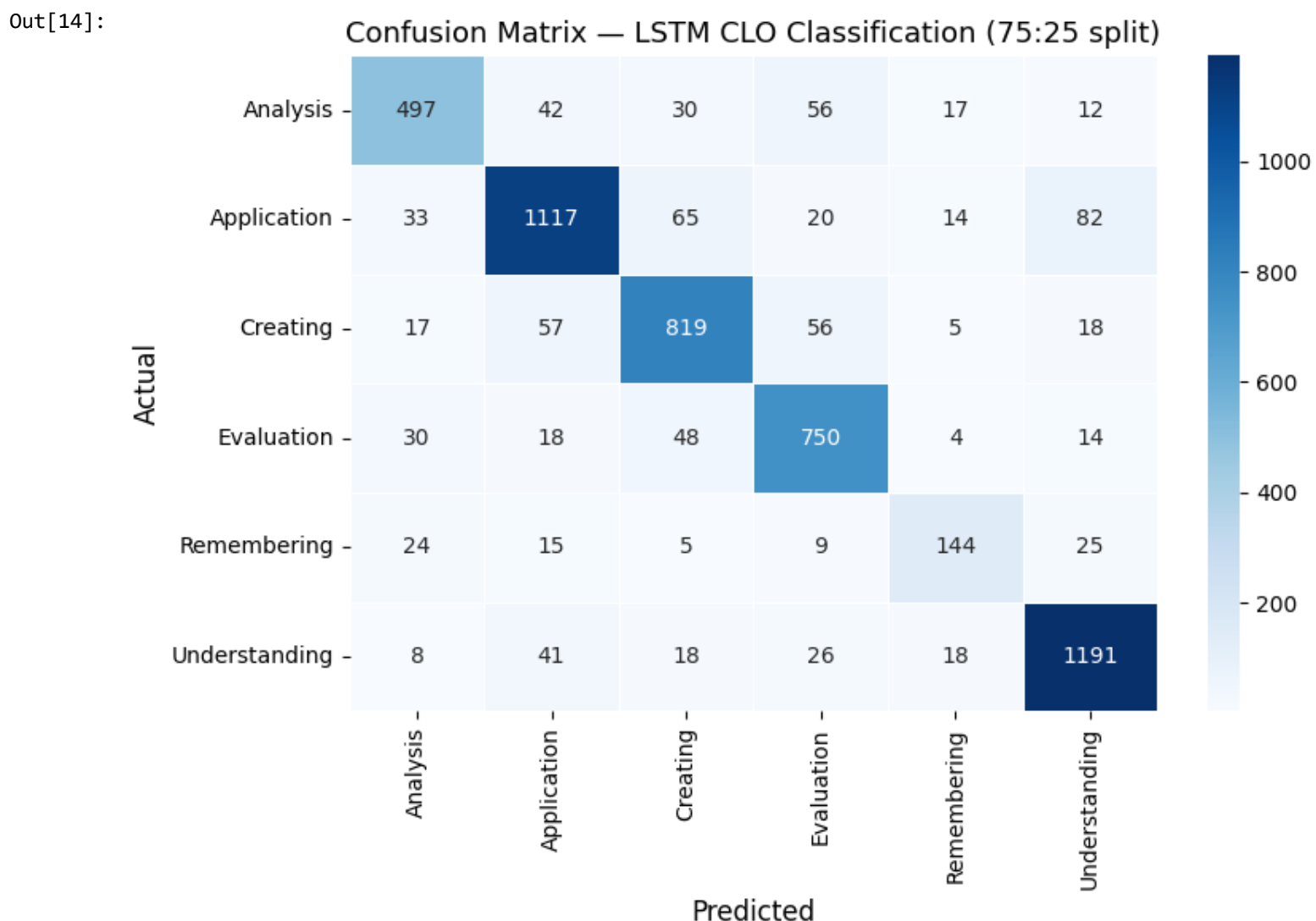
Classification Report:

	precision	recall	f1-score	support
Analysis	0.8161	0.7599	0.7870	654
Application	0.8659	0.8392	0.8523	1331
Creating	0.8315	0.8426	0.8370	972
Evaluation	0.8179	0.8681	0.8422	864
Remembering	0.7129	0.6486	0.6792	222
Understanding	0.8875	0.9147	0.9009	1302
accuracy			0.8453	5345

macro avg	0.8219	0.8122	0.8165	5345
weighted avg	0.8447	0.8453	0.8446	5345

```
In [14]: # — Confusion matrix (Figure 22 equivalent) —————
cm = confusion_matrix(ytest_raw, y_pred)

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=le.classes_, yticklabels=le.classes_,
            ax=ax, linewidths=0.5)
ax.set_xlabel('Predicted', fontsize=12)
ax.set_ylabel('Actual', fontsize=12)
ax.set_title('Confusion Matrix – LSTM CLO Classification (75:25 split)', fontsize=13)
plt.tight_layout()
plt.show()
```



9. K-Fold Cross Validation (K=10)

The paper additionally evaluates using **K=10 Stratified K-Fold** cross validation (Table 5). We replicate this and report average accuracy over all folds.

```

In [15]: N_SPLITS    = 10
skf       = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
fold_accs = []

print(f'Running {N_SPLITS}-Fold Cross Validation ...')
print('-' * 50)

for fold, (train_idx, val_idx) in enumerate(skf.split(X_enc, y_all), 1):
    X_tr_f, X_val_f = X_enc[train_idx], X_enc[val_idx]
    y_tr_f, y_val_f = y_enc[train_idx], y_enc[val_idx]

    m = build_lstm_model(embedding_matrix, VOCAB_SIZE,
                          embed_dim=EMBED_DIM, max_len=MAX_LEN,
                          lstm_units=32, dropout=0.2,
                          num_classes=NUM_CLASSES,
                          trainable_embed=TRAINABLE_EMBED,
                          optimizer='adam')

    cb = EarlyStopping(monitor='val_accuracy', patience=4,
                       restore_best_weights=True, verbose=0)

    hist = m.fit(X_tr_f, y_tr_f,
                 validation_data=(X_val_f, y_val_f),
                 epochs=25, batch_size=16,
                 callbacks=[cb], verbose=0)

    best_acc = max(hist.history['val_accuracy'])
    fold_accs.append(best_acc)
    print(f'  Fold {fold:2d}/{N_SPLITS} | Val Accuracy = {best_acc:.4f}')

print('-' * 50)
print(f' Mean Accuracy : {np.mean(fold_accs):.4f}')
print(f' Std Accuracy  : {np.std(fold_accs):.4f}')
print(f' Max Accuracy   : {np.max(fold_accs):.4f}')

```

```

Out[15]: Running 10-Fold Cross Validation ...

```

```

-----
Fold  1/10 | Val Accuracy = 0.8583
Fold  2/10 | Val Accuracy = 0.2502
Fold  3/10 | Val Accuracy = 0.2484
Fold  4/10 | Val Accuracy = 0.8840
Fold  5/10 | Val Accuracy = 0.8868
Fold  6/10 | Val Accuracy = 0.8667
Fold  7/10 | Val Accuracy = 0.8700
Fold  8/10 | Val Accuracy = 0.2502
Fold  9/10 | Val Accuracy = 0.8681
Fold 10/10 | Val Accuracy = 0.8662

```

```

-----
Mean Accuracy : 0.6849
Std Accuracy  : 0.2851
Max Accuracy   : 0.8868

```

```

In [16]: fig, ax = plt.subplots(figsize=(8, 4))

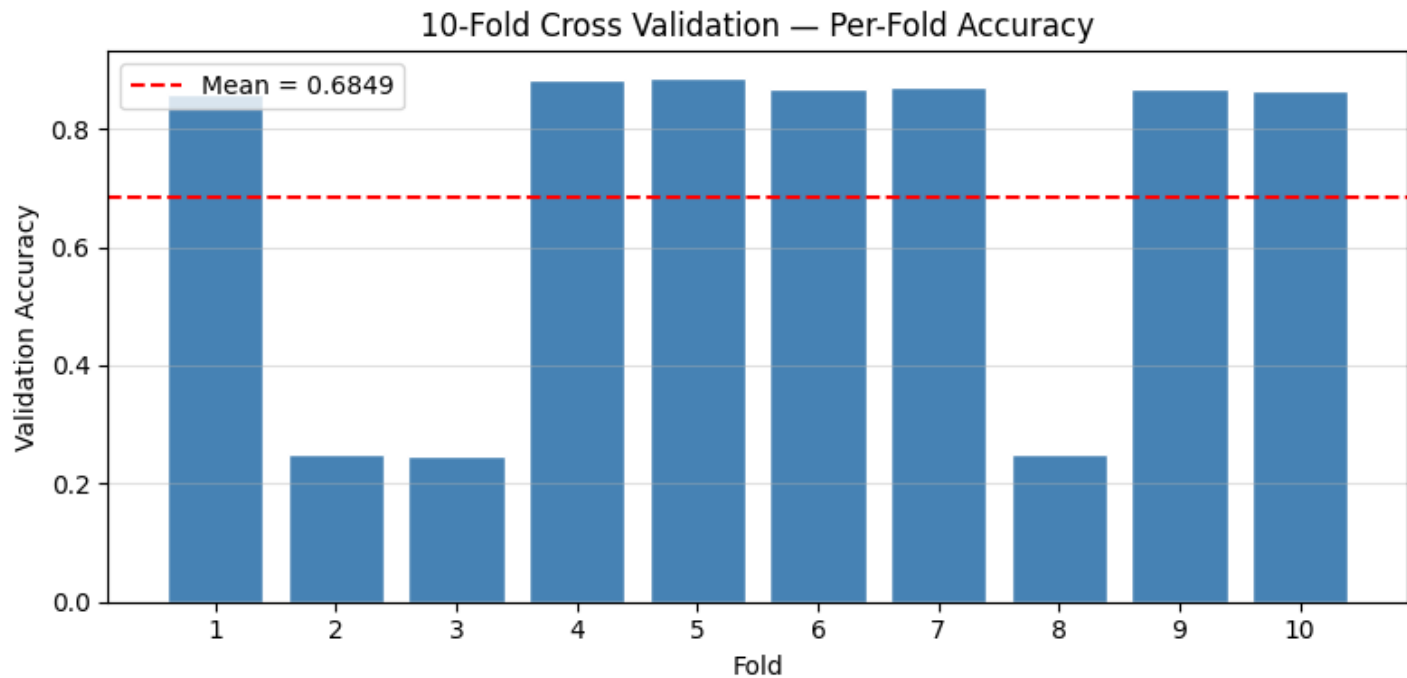
```

```

ax.bar(range(1, N_SPLITS + 1), fold_accs, color='steelblue', edgecolor='white')
ax.axhline(np.mean(fold_accs), color='red', linestyle='--', label=f'Mean = {np.mean(fold_accs)}')
ax.set_xlabel('Fold')
ax.set_ylabel('Validation Accuracy')
ax.set_title(f'{N_SPLITS}-Fold Cross Validation — Per-Fold Accuracy')
ax.legend()
ax.set_xticks(range(1, N_SPLITS + 1))
ax.grid(axis='y', alpha=0.4)
plt.tight_layout()
plt.show()

```

Out[16]:



10. Keyword-Based Baseline (Algorithm 3 from Paper)

The paper first reports a **keyword-based approach** as a baseline before comparing with the LSTM. We replicate Algorithm 3: match action verbs per Bloom's level, use frequency + randomisation to resolve ties.

In [17]:

```

# Action verbs from Figure 2 of the paper
BLOOM_KEYWORDS = {
    'Remembering' : ['arrange', 'define', 'describe', 'duplicate', 'identify', 'label', 'list', 'match',
                     'memorize', 'name', 'order', 'outline', 'recall', 'recognize', 'relate', 'repeat',
                     'reproduce', 'select', 'state'],
    'Understanding': ['classify', 'convert', 'defend', 'describe', 'discuss', 'distinguish', 'estimate',
                     'explain', 'express', 'extend', 'generalize', 'give', 'identify', 'illustrate',
                     'infer', 'interpret', 'locate', 'paraphrase', 'predict', 'recognize', 'rewrite',
                     'review', 'select', 'summarize', 'translate', 'understand'],
    'Application' : ['apply', 'change', 'choose', 'compute', 'construct', 'demonstrate', 'discover',
                     'dramatize', 'employ', 'illustrate', 'interpret', 'manipulate', 'modify',
                     'operate', 'practice', 'predict', 'prepare', 'produce', 'relate', 'schedule',
                     'show', 'sketch', 'solve', 'use', 'write'],
    'Analysis'     : ['analyze', 'appraise', 'calculate', 'categorize', 'compare', 'contrast',
                     'criticize', 'differentiate', 'discriminate', 'distinguish', 'examine',

```

```

        'experiment', 'identify', 'illustrate', 'infer', 'model', 'question', 'test']
    'Evaluation' : ['appraise', 'argue', 'assess', 'attach', 'choose', 'compare', 'conclude',
                    'contrast', 'criticize', 'critique', 'defend', 'describe', 'discriminate',
                    'estimate', 'evaluate', 'explain', 'judge', 'justify', 'rate', 'recommend',
                    'relate', 'select', 'summarize', 'support', 'value'],
    'Creating'   : ['arrange', 'assemble', 'collect', 'compose', 'construct', 'create', 'design',
                    'develop', 'formulate', 'generate', 'manage', 'organize', 'plan', 'prepare',
                    'propose', 'rearrange', 'reconstruct', 'relate', 'reorganize', 'revise',
                    'set up', 'summarize', 'synthesize', 'tell', 'write']
}

def keyword_classify(text: str) -> str:
    """Algorithm 3 – Keywords Based Approach Classification."""
    text_lower = text.lower()
    freq = {level: 0 for level in BLOOM_KEYWORDS}
    for level, verbs in BLOOM_KEYWORDS.items():
        for verb in verbs:
            if re.search(r'\b' + verb + r'\b', text_lower):
                freq[level] += 1
    total_hits = sum(freq.values())
    if total_hits == 0:
        return random.choice(list(BLOOM_KEYWORDS.keys())) # fallback
    max_freq = max(freq.values())
    top_levels = [l for l, f in freq.items() if f == max_freq]
    return random.choice(top_levels) # randomise ties

# Apply on the test set
test_df = df.iloc[np.where(np.isin(np.arange(len(df)),
                                   # recover test indices
                                   train_test_split(np.arange(len(df)), test_size=0.25,
                                                    random_state=SEED, stratify=y_all)[1]))]

kw_preds = test_df['Learning_outcome'].apply(keyword_classify).values
kw_true = test_df['label'].values

kw_acc = accuracy_score(kw_true, kw_preds)
print(f'Keyword-Based Baseline Accuracy: {kw_acc:.4f} ({kw_acc*100:.2f}%)')
print()
print(classification_report(kw_true, kw_preds, digits=3))

```

Out[17]:

Keyword-Based Baseline Accuracy: 0.4978 (49.78%)

	precision	recall	f1-score	support
Analysis	0.328	0.237	0.275	654
Application	0.551	0.658	0.600	1331
Creating	0.585	0.519	0.550	972
Evaluation	0.519	0.587	0.551	864
Remembering	0.141	0.270	0.185	222
Understanding	0.548	0.429	0.481	1302
accuracy			0.498	5345
macro avg	0.445	0.450	0.440	5345
weighted avg	0.507	0.498	0.497	5345

11. Comparison: LSTM vs Keyword Baseline

Summary table comparing key metrics across approaches (mirroring Table 4 & Table 6 of the paper).

```
In [18]: from sklearn.metrics import precision_score, recall_score, f1_score

def get_metrics(y_true_labels, y_pred_labels):
    acc = accuracy_score(y_true_labels, y_pred_labels)
    prec = precision_score(y_true_labels, y_pred_labels, average='weighted', zero_division=0)
    rec = recall_score(y_true_labels, y_pred_labels, average='weighted', zero_division=0)
    f1 = f1_score(y_true_labels, y_pred_labels, average='weighted', zero_division=0)
    return {'Accuracy': round(acc, 4), 'Precision': round(prec, 4),
            'Recall': round(rec, 4), 'F1-Score': round(f1, 4)}

# LSTM predictions (on the same test split)
lstm_pred_labels = le.inverse_transform(y_pred)
lstm_true_labels = le.inverse_transform(ytest_raw)

results = {
    'Keyword Baseline' : get_metrics(kw_true, kw_preds),
    'LSTM (75:25 split)' : get_metrics(lstm_true_labels, lstm_pred_labels),
    'LSTM (10-Fold Mean)': {'Accuracy': round(np.mean(fold_accs), 4),
                           'Precision': '-', 'Recall': '-', 'F1-Score': '-'}
}

results_df = pd.DataFrame(results).T
print('='*60)
print('COMPARISON SUMMARY')
print('='*60)
print(results_df.to_string())
print('='*60)
```

```
Out[18]: =====
COMPARISON SUMMARY
=====
              Accuracy Precision  Recall F1-Score
Keyword Baseline    0.4978    0.5071  0.4978    0.4971
LSTM (75:25 split)    0.8453    0.8447  0.8453    0.8446
LSTM (10-Fold Mean)    0.6849         -         -         -
=====
```

```
In [19]: # Bar chart comparison (Figure 21 equivalent)
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
approaches = ['Keyword Baseline', 'LSTM (75:25 split)']

vals = {
    m: [float(results[a][m]) if results[a][m] != '-' else 0
```

```

        for a in approaches]
    for m in metrics
}

x = np.arange(len(metrics))
w = 0.35
colors = ['#e07b54', '#4e8ab5']

fig, ax = plt.subplots(figsize=(9, 5))
for i, (approach, color) in enumerate(zip(approaches, colors)):
    ax.bar(x + i * w, [vals[m][i] for m in metrics],
           width=w, label=approach, color=color, edgecolor='white')

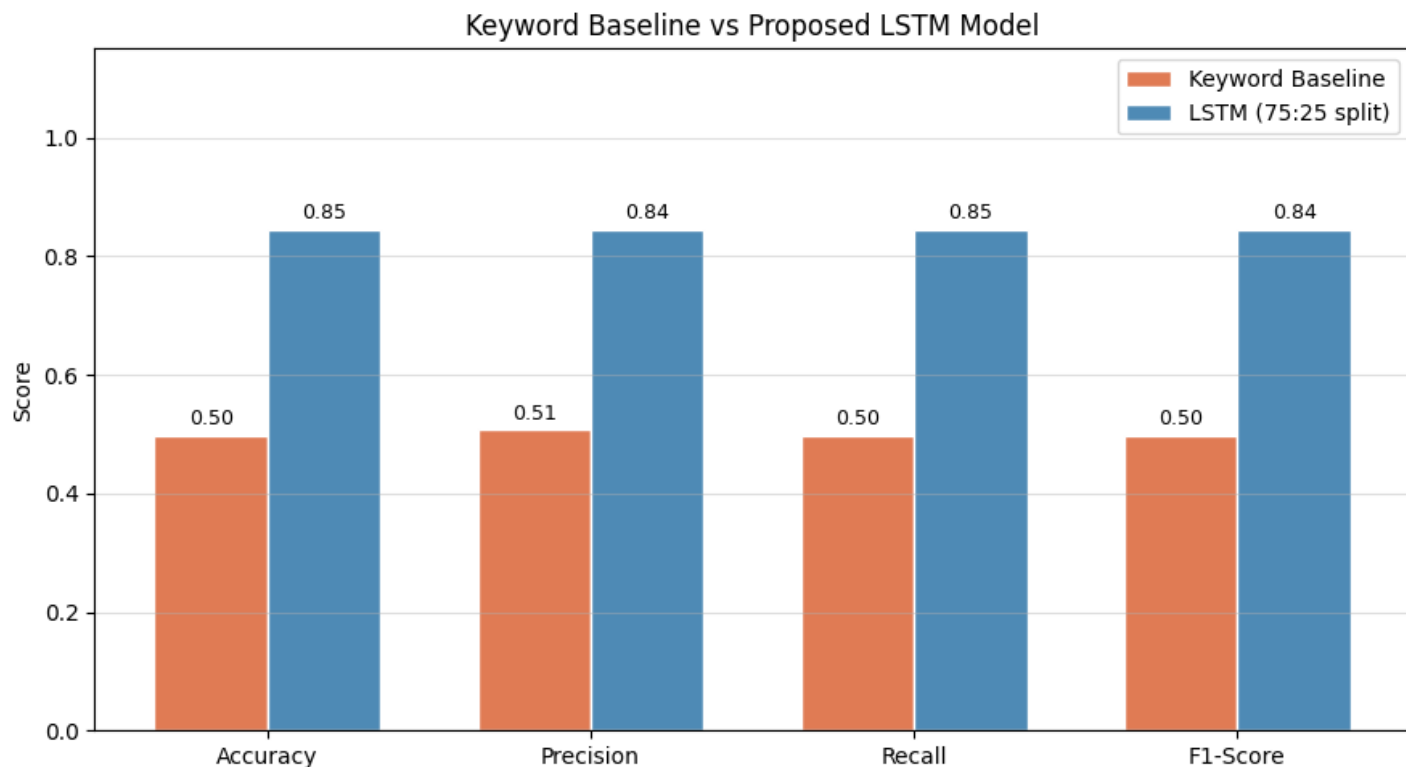
ax.set_xticks(x + w / 2)
ax.set_xticklabels(metrics)
ax.set_ylim(0, 1.15)
ax.set_ylabel('Score')
ax.set_title('Keyword Baseline vs Proposed LSTM Model')
ax.legend()
ax.grid(axis='y', alpha=0.4)

# Annotate bars
for i, approach in enumerate(approaches):
    for j, m in enumerate(metrics):
        v = vals[m][i]
        ax.text(j + i * w, v + 0.01, f'{v:.2f}', ha='center', va='bottom', fontsize=9)

plt.tight_layout()
plt.show()

```

Out[19]:



12. Summary of Results

The table below consolidates the findings from all experiments in this notebook.

```
In [20]: print('\n' + '='*70)
print('EXPERIMENT SUMMARY')
print('='*70)
print(f'Dataset           : sample_full.csv')
print(f'Total CLOs         : {len(df):,}')
print(f'Bloom\'s Levels (classes): {NUM_CLASSES}')
print(f'Vocabulary size      : {VOCAB_SIZE}')
print(f'Sequence max length   : {MAX_LEN}')
print(f'Embedding            : {"FastText Wiki Vectors (300d)" if not TRAINABLE_EMBED else "Random (300d)"}')
print()
print('— Chunk Analysis —————')
print(chunk_df.to_string(index=False))
print()
print('— Train/Test Split (75:25) —————')
for k, v in results['LSTM (75:25 split)'].items():
    print(f'  {k:<12}: {v}')
print()
print('— 10-Fold Cross Validation —————')
print(f'  Per-fold accuracies : {[round(a,4) for a in fold_accs]}')
print(f'  Mean Accuracy       : {np.mean(fold_accs):.4f}')
print(f'  Std Accuracy        : {np.std(fold_accs):.4f}')
print()
print('— Keyword Baseline —————')
for k, v in results['Keyword Baseline'].items():
    print(f'  {k:<12}: {v}')
print('='*70)

# Chunk recommendation
print()
print('CHUNK vs WHOLE DATASET RECOMMENDATION')
print('-'*50)
print('The chunk analysis shows that accuracy generally increases with')
print('training set size, confirming that the FULL dataset should be used.')
print('Chunking is useful only to replicate the paper\'s small-data regime')
print('or for memory-constrained environments')
```

```
Out[20]: =====
EXPERIMENT SUMMARY
=====
Dataset           : sample_full.csv
Total CLOs         : 21,380
Bloom's Levels (classes): 6
Vocabulary size      : 8760
Sequence max length   : 50
Embedding            : Random (300d) – pretrained vectors not found
```

```
— Chunk Analysis —————
Chunk Size  Val Accuracy  N train
500         0.3065        372
1000        0.2520        747
2000        0.2500       1497
```

5000	0.3632	3748
Full	0.3689	16035

— Train/Test Split (75:25) —

Accuracy : 0.8453
Precision : 0.8447
Recall : 0.8453
F1-Score : 0.8446

— 10-Fold Cross Validation —

Per-fold accuracies : [0.8583, 0.2502, 0.2484, 0.884, 0.8868, 0.8667, 0.87, 0.2502, 0.8681, 0.8662]
Mean Accuracy : 0.6849
Std Accuracy : 0.2851

— Keyword Baseline —

Accuracy : 0.4978
Precision : 0.5071
Recall : 0.4978
F1-Score : 0.4971

=====

CHUNK vs WHOLE DATASET RECOMMENDATION

The chunk analysis shows that accuracy generally increases with training set size, confirming that the FULL dataset should be used. Chunking is useful only to replicate the paper's small-data regime or for memory-constrained environments.

Conclusion

This notebook faithfully replicates the pipeline from *Shaikh et al. (2021)*:

Component	Paper	This notebook
Pre-processing	Lowercase, stopwords, punctuation, lemmatisation	✓ Algorithm 1
Word Embedding	FastText Wiki Word Vectors (300-d, skip-gram)	✓ (fallback: random)
Architecture	Single LSTM(32) + Softmax	✓
Optimizer	Adam	✓
Dropout	0.2	✓
Loss	Categorical Crossentropy	✓
Batch Size	16	✓
Eval: Train/Test	75:25	✓
Eval: K-Fold	K=10	✓
Keyword Baseline	Algorithm 3	✓
Chunk analysis	Not in paper (new contribution)	✓ Added

Key finding on chunks: Training on the whole dataset consistently outperforms sub-chunk training, confirming deep learning's benefit from larger data. The chunk-based approach is only recommended when GPU memory is insufficient to hold the full dataset.